CS496 AI Engineering, Mediterranean Institute of Technology, Fall 2026
Dr Abdeldjalil Labed

Project 2: The Librarian

Nadia Ben Njima, Ahmed Brini, Rahma Sbei, Mariem Lahmer

Repository: https://github.com/Rahma-sbei/CS496-project2-The-Librarian

Live app: https://the-librarian-6wzd.onrender.com (may take a minute to wake up)

Summary. The Librarian answers questions about the Streamlit Python library using only Streamlit's documentation, closed GitHub issues and release notes, cites the passage behind each claim, and says "I don't know" when the passages don't contain the answer. Hybrid search (keyword + vector) finds the right passage in the top 5 for **80%** of our 100 test questions, against 69% for keywords alone and 70% for vectors alone. Answers are correct on **86.6%** of all 112 questions and **98.5%** of their claims are supported by a retrieved passage. A question costs **$0.00031** and takes 1.3 s (p95 2.3 s). For Week 8, sending 2 passages instead of 5 cut the cost by 53%, but lost 10.3 points of accuracy, more than the 2 allowed, so the service keeps 5.

1. The corpus and its users

We chose an open-source project's docs, issues and releases (one of the suggested corpora) and picked **Streamlit**, the Python library for building data apps, because our class already used it, so we know its users and their questions. It passes the five tests from the brief:

Test	Streamlit
Real	The documentation and issue tracker developers use today (snapshot taken 2026-10-25).
Real users	Streamlit developers, starting with our classmates, who hit the same errors and version questions.
Allowed to use	Public, Apache 2.0. Personal data removed: usernames, emails, tokens and home paths are replaced by placeholders, author logins are never stored.
Big enough to need search	404 docs pages, 418 closed issues with discussion, 60 releases: 5 MB of text, 3,562 chunks. It cannot fit in one prompt.
Checkable	Answers can be quoted word for word: our 100 answerable questions each point to an exact passage.

The three sources play different roles: the **docs** are authoritative but generic; **issues** hold real error messages and maintainers' workarounds; **releases** hold version facts ("which release added X?") that a language model cannot have memorised. The corpus and index are committed as a frozen snapshot so the evaluation's exact quotes stay valid and the project runs from a clean clone without a GitHub token.

2. Pipeline design

The system has an offline part, run once, and an online part, run for every question:

1. **Offline.** `build_corpus.py` clones the docs and calls the GitHub API for issues and releases, removing personal data; `index_corpus.py` cleans and chunks the text, embeds each chunk and builds a keyword index, saved as flat files in `index/`.
2. **Online.** The question (optionally rewritten) goes to keyword search and vector search, the two top-50 lists are fused, an optional filter keeps only docs, issues or releases, and the top 5 chunks go to `gpt-4o-mini`, which answers with `[n]` citations or "I don't know". A FastAPI service and a web page wrap this and log latency and cost per question.

Chunking. Each source is cut differently. Docs are split at headings, then packed paragraph by paragraph to about 250 words with a 40-word overlap; code blocks are never split so code stays runnable. The docs are MDX (Markdown mixed with website components), so we strip the markup first and merge chunks under 40 words into a neighbour, which took the index from 4,007 noisy chunks to 3,562 clean ones. Issues keep their description and discussion apart, and each comment is tagged with its author's role (a maintainer's reply weighs more). Each chunk keeps its title, section, URL, date and type, and is embedded as "title > section + text" so it carries its heading's context. 250 words keeps most docs sections whole while five chunks stay around 1,700 tokens. Our scripts can score any rebuilt index, but we ran out of time to compare 150- and 400-word chunks, so this size is a reasoned choice, not a measured one.

Index: flat files. 3,562 vectors of 384 numbers take 5 MB, so comparing the question with every chunk is one matrix product (a few milliseconds). A vector database would add a server and a bill for no gain at this size; the index ships inside the app and rebuilds in about 5 minutes. Trade-offs: no incremental updates, and filtering is done in Python. The embedding model, `BAAI/bge-small-en-v1.5`, runs free on a CPU.

Retrieval. Keyword search (BM25) is strong on exact strings like error messages and function names; vector search is strong on paraphrases. Each returns 50 candidates and reciprocal rank fusion merges them, rewarding chunks both rank highly. Users can restrict the search to docs, issues or release notes (metadata filter), and can ask the model to rewrite a vague question first. **Reranking:** we measured a cross-encoder (`ms-marco-MiniLM-L-6-v2`) on the top 30. It improves the first result but not the top 5 the model reads, and adds about 2,098 ms per question, so it is not used in the service (section 3).

Generation. `gpt-4o-mini` (temperature 0) receives the numbered passages and three rules: end every factual sentence with its source number, use nothing outside the sources, and reply exactly "I don't know." if they don't answer the question. The code turns the `[n]` markers into links and flags numbers that don't exist. We also built a pre-filter that refuses when the best vector similarity is low, but at a threshold of 0.75 it refused 92% of unanswerable questions *and* 15% of answerable ones, so the service leaves refusing to the model.

3. Retrieval results and ablation

Evaluation set. 100 answerable questions, each with a reference answer and one or more exact source passages (41 accept more than one correct page), plus 12 unanswerable questions about other libraries or features Streamlit doesn't have. Question types: how-to 47, conceptual 17, version 15, error-message 13, multi-part 8. Sources of the answerable questions: 86 docs, 11 issues, 3 releases. A stratified 20% test split is kept apart. A question counts as found at *k* if any of the top *k* chunks contains one of its passages; the scripts recompute this for whichever index they test, so a re-chunked index is scored the same way.

Search	R@1	R@3	R@5	R@10	R@20	Doc R@5	MRR@10	p50 ms	p95 ms
Keyword only (BM25)	0.29	0.55	0.69	0.78	0.89	0.74	0.44	13	22
Vector only	0.47	0.67	0.70	0.82	0.85	0.79	0.58	23	28
Hybrid (RRF)	0.38	0.69	0.80	0.86	0.90	0.84	0.55	41	58
Hybrid + reranker	0.43	0.74	0.79	0.88	0.93	0.83	0.60	2,098	2,339

What the ablation shows. Neither search alone is enough: keywords miss paraphrases and vectors miss exact names, and fusing them lifts recall@5 from 0.70 to **0.80**. The reranker lifts recall@1 (0.38 → 0.43) and recall@20, but not recall@5 (0.79), the part the model actually reads, while search time grows from 41 ms to 2,098 ms on a laptop CPU. Hybrid without reranking is the best trade-off. Searching costs nothing per question: both indexes run locally.

Recall@5 by question type:

Question type	n	Keyword only (BM25)	Vector only	Hybrid (RRF)	Hybrid + reranker
how-to	47	0.70	0.66	0.79	0.79
conceptual	17	0.65	0.76	0.76	0.76
multi-part	8	0.75	1.00	1.00	1.00
error-message	13	0.85	0.69	0.85	0.85
version	15	0.53	0.60	0.73	0.67

4. Generation results: faithfulness and correctness

Retrieval and generation are measured separately: the generation script runs the real service code on all 112 questions, then a second model call (the "judge", `gpt-4o-mini`) splits each answer into claims, checks each claim against the passages and against the cited passage, and grades the answer against the reference answer. Unanswerable questions score 1 only if the answer is "I don't know".

Generation metric	Result
Accuracy, all 112 questions (headline for Week 8)	86.6%
Correctness on the 100 answerable (yes = 1, partial = 0.5)	85.0%
Faithfulness: claims stated by a retrieved passage	98.5%
Answers in which every claim is supported	95.8%
Citation accuracy: the cited passage states the claim	99.1%
Claims that carry a citation	89.2%
Answers citing a source number that doesn't exist	0.0%
"I don't know" on the 12 unanswerable (want 100%)	100.0%
"I don't know" on answerable questions (want 0%)	5.0%
Gold passage among the 5 chunks sent to the model	80.0%

Question type	n	Correctness
conceptual	17	97%
error-message	13	96%
how-to	47	80%
multi-part	8	88%
unanswerable	12	100%
version	15	77%

We read the judge's "partial" and "wrong" verdicts. It is right on the clear errors and strict on answers that are correct but leave out a detail of the reference (marked "partial", 0.5), so accuracy is a conservative estimate. All judge calls for this report cost $0.08.

Reading the numbers. When the right passage reaches the model it rarely goes beyond it: 98.5% of claims are backed by a passage, and when a claim cites a source, that source states it 99.1% of the time. The weak spot is *coverage of citations*: only 89% of claims carry a citation even though the prompt asks for one on every factual sentence. "I don't know" works in both directions: 100% on unanswerable questions and 5% on answerable ones. Of the 25 answerable questions not fully correct, 5 were "I don't know" and 10 never received the right passage (it was among the 5 chunks for 80% of questions). Example: asked for collapsible content, the model was not given the `st.expander` passage and faithfully described embedding custom HTML instead: faithful, but wrong. Better search would help more than a better prompt.

5. Week 8: making it cheaper

Almost all of the cost (85%) is input tokens, and almost all of the input is the five passages; search itself is free. Sending fewer passages is the first saving the brief suggests, so we measured 5, 3 and 2 with the same questions, model and judge (latency is end to end, including the call to OpenAI):

Passages	Input tokens	Cost	Δ cost	p50	p95	Δ p95	Accuracy	Δ acc.	Faithful	Gold sent
5 (kept)	1,733	$0.000305	–	1.34 s	2.34 s	–	86.6%	–	98.5%	80%
3	1,038	$0.000197	-35%	1.19 s	2.42 s	+3%	80.4%	-6.2 pts	99.0%	69%
2	708	$0.000144	-53%	1.21 s	2.12 s	-9%	76.3%	-10.3 pts	98.8%	59%

The cost target was reached, the accuracy limit was not. Two passages cut the cost by 53%, but accuracy fell from 86.6% to 76.3% (10.3 points); three passages cut 35% for 6.2 points. The model did not get worse at using its passages (faithfulness stays near 99%); it more often did not receive the right one: the gold passage was sent for 80% of questions with 5, 69% with 3 and 59% with 2. Our ranking finds the right passage within the top 5 but rarely puts it first. Latency barely moves because the model call dominates it, so a 50% p95 cut is not reachable this way. **Decision:** the service keeps 5 passages.

Two changes do save at no accuracy cost: we **skip the reranker** (about 2,098 ms per question for no recall@5 gain), and the service **caches repeated questions**, answered with no search, no model call and no cost. Next we would try sending shorter passages instead of fewer (only the paragraphs most similar to the question), a better top-ranking so 2 or 3 passages suffice, or the cheaper `gpt-4.1-nano`. Details: `optimisation.md`.

6. Known failure cases and cost model

- **API reference pages are empty** in the corpus: the website fills in each function's description at build time, and our Markdown only has a placeholder tag. Questions about a function's parameters depend on other pages.
- **Very long chunks**: code blocks and pasted logs are never split; 17 chunks exceed 400 words (one is 3,923), and the embedding model only reads the first 512 tokens.
- **Tables** are flattened to rows, so column meaning can be lost; **cross-references** are kept as text but not followed, so an answer spread over two pages needs both retrieved.
- **Issues are noisy**: a thread can contain a wrong first answer corrected later.
- **Missing citations** on some sentences, and a judge from the same model family.
- **Docs pages have no date**, so "since version X" relies on release notes.

Cost model. Embedding, search and hosting the index are free (local CPU); the only per-question cost is the language model at $0.15 per million input tokens and $0.60 per million output tokens:

Configuration	Per question	Per 1,000	Per month at 100/day	Per month at 10,000/day (100×)
5 passages (deployed)	$0.00031	$0.31	$0.92	$91.62
2 passages	$0.00014	$0.14	$0.43	$43.23

The service also caps each visitor at 10 questions a minute and the whole service at 300 a day, so a public URL cannot run up the bill. Hosting is free: the service runs on Render's free plan (512 MB), which it fits after we moved question embedding from PyTorch to ONNX (section 7). At 100× traffic the model cost stays small; the limit becomes that plan's single small instance.

7. Postmortem: what went wrong and what we learned

- **Our first evaluation set measured our labelling, not our search.** It was written without the corpus open (invented file names, passages from memory); a script then re-attached each question to the most *similar* chunk, often an unrelated issue (the `st.stop()` question pointed at a dialog bug).

Recall@5 read 0.42. We rebuilt it by hand against the index: 69 questions re-grounded, several correct pages allowed, answers rewritten from the passages, and 31 trivial "which widget does X" questions replaced by error, version and issue questions. Recall@5 rose to 0.80 with the same search. *Lesson: read what was retrieved before blaming the search.*

- **A passing check is not a quality check.** Our validator said "0 problems" while a third of the answers were not supported by their passages; it only checked that passages existed.
- **Read the data before building on it.** Website markup in the docs, empty API pages and a 3,923-word chunk were all found by opening files. A Windows permission error, an issue collector that kept only 149 of 600 issues, and a missing `openai` requirement were found by running the code from scratch.
- **Resources and hosting.** Two evaluations at once crashed with an out-of-memory error (each loads a 600 MB model). Our planned free host, Hugging Face Spaces, now charges for Docker apps (`402 Payment Required`), and the remaining free hosts give 512 MB of RAM while the service used 640 MB. We now embed questions with the same model through ONNX (`fastembed`): 290 MB, question vectors matching PyTorch's (cosine ≥ 0.99999) and identical recall at every k, so the service runs on Render's free plan and our evaluation still describes it.
- **Week 8.** Fewer passages met the cost target but not the accuracy limit: we optimised the cheapest lever before improving the ranking it depends on.
- **Process.** We started measuring too late; next time the evaluation set and a recall script come first. The chunk-size comparison was planned but not finished. The full list of 21 problems and fixes is in `postmortem.md`.